

BAB 2 Dasar JavaScript untuk React Native

2.1 Tujuan Pembelajaran

1. Memahami peran JavaScript dalam pengembangan aplikasi React Native.
2. Menjelaskan konsep dasar JavaScript yang digunakan dalam React Native.
3. Menggunakan variabel, tipe data, operator, fungsi, dan array dalam konteks React Native.
4. Menerapkan konsep JavaScript dasar untuk membangun logika sederhana pada aplikasi mobile.
5. Menyiapkan pemahaman JavaScript sebagai fondasi sebelum mempelajari komponen dan state React Native.

2.2 Dasar Teori

React Native menggunakan *JavaScript* sebagai bahasa utama untuk membangun antarmuka dan logika aplikasi *mobile*. Berbeda dengan *JavaScript* pada web yang berjalan di *browser*, *JavaScript* pada *React Native* dijalankan oleh *JavaScript engine* (seperti *Hermes* atau *JavaScriptCore*) dan berkomunikasi dengan komponen *native* Android dan iOS.

2.2.1 Variable

JavaScript menyediakan tiga cara utama untuk mendeklarasikan variabel, yaitu menggunakan *let* dan *const*.

- *let* : untuk data yang dapat berubah.
- *const* : untuk data yang tidak berubah.
- *var* : tipe data ini sudah tidak direkomendasikan untuk digunakan pada ES6.

```
var nama = "Febry";  
let nim = 3120200907;  
const kelas = "TI-24-KA";
```

2.2.2 Tipe Data Dasar

Beberapa tipe data yang sering digunakan dalam React Native:

Type Data	Contoh
String	"Mobile Programming"
Number	10, 3.14
Boolean	true, false
Object	{ nama: "Budi" }
Array	["Android", "iOS"]

Tabel 2.1 Table Tipe Data

Contoh penggunaannya seperti dibawah ini:

```

1  const class_name = 'Praktikum Pemograman Perangkat Bergerak';
2  const total_student = 28;
3  const lecturer = {name: 'Febry Damatraseta Fairuz', nim: '3120200907'};
4  const status_class = true;
5  const intake = [2023, 2024];
6  const students = [
7    {name: 'Anton', nim: '3120200901'},
8    {name: 'Isnan', nim: '3120200902'},
9    {name: 'Edi', nim: '3120200903'},
10 ]

```

Penjelasan:

- variable `class_name` : menyimpan nilai String.
- variable `total_student` : menyimpan nilai number.
- variable `lecturer` : menyimpan nilai object.
- variable `status_class` : menyimpan nilai boolean.
- variable `intake` : menyimpan nilai array.
- variable `students` : menyimpan nilai array object.

2.2.3 Operator

Operator digunakan untuk melakukan operasi pada data, seperti dibawah ini:

```

1  let a = 10;
2  let b = 5;
3  let hasil_penjumlahan = a + b;
4  let hasil_pengurangan = a - b;
5  let hasil_perkalian = a * b;
6  let hasil_pembagian = a / b;
7  let hasil_modulus = a % b;

```

2.2.4 Fungsi

Fungsi digunakan untuk mengelompokkan logika program.

```

function greetings(){
  return `Welcome to ${class_name} class with ${total_student} students.`;
}

```

```

function penjumlahan(a, b) {
  return a + b;
}

```

Atau menggunakan **arrow function** (lebih sering dipakai di React Native):

```

const greetings = () =>{
  return `Welcome to ${class_name} class with ${total_student} students.`;
}

```

```
const penjumlahan = (a, b) => {  
  return a + b;  
};
```

2.2.5 Array dan Object

Dalam JavaScript, **array** dan **object** digunakan untuk menyimpan data yang lebih kompleks dibandingkan variabel tunggal. Keduanya memiliki karakteristik dan tujuan penggunaan yang berbeda.

1. Array

a) Data Disimpan dalam Urutan (Index)

- Setiap elemen memiliki index
- Index dimulai dari 0

```
const intake = [2023, 2024];
```

Variable intake memiliki 2 buah value number. Dimana index **ke-0** adalah **2023** dan **index ke-1** adalah **2024**.

b) Tidak Menggunakan Key Bernama

- Akses data menggunakan index angka

Jika ingin menampilkan value dari variable array, cukup menginisialisasi index yang diinginkan misal, `intake[0]` akan menampilkan output 2023.

c) Baik digunakan untuk data yang sejenis

```
const intake = [2023, 2024];  
const courses = ['Matematika', 'Kalkulus'];
```

Dalam menggunakan data yang sejenis, tidak boleh memiliki nilai yang berbeda-beda, seperti dalam 1 array terdapat number, string ataupun object.

d) Jumlah data dapat berubah

Bentuk array memiliki sebuah function yang diperuntukan untuk menambah data, yaitu function `push()`.

```
courses.push('Fisika');
```

2. Object

Object adalah struktur data yang digunakan untuk menyimpan data dengan pasangan **key** dan **value**.

1. Harus memiliki Key dan Value

- key: nama properti
- value: isi data

```
const lecturer = {  
  name: 'Febry Damatraseta Fairuz'  
};
```

Untuk memiliki nilai dalam bentuk object selain harus memiliki key dan value, harus menggunakan simbol **kurung kurawal** buka dan tutup, dan simbol titik dua (:) sebagai penanda antara key dan value, seperti **{ key : value }**. Pada contoh variable lecturer, disana memiliki 1 bentuk object yang terdiri dari:

- key → name
- value → 'Febry Damatraseta Fairuz'

Jika ingin menambahkan data lebih dari 1 key maka gunakan simbol koma (,) sebagai pemisah, seperti dibawah ini:

```
const lecturer = {  
  name: 'Febry Damatraseta Fairuz',  
  nim: '3120200907'  
};
```

2. Key bersifat unik

Tidak boleh ada key yang sama dalam satu object. Seperti dibawah ini:

```
{  
  name: 'Febry Damatraseta Fairuz',  
  name: 'Anton Sukamto'  
}
```

3. Value bisa berbagai tipe data

```
{  
  name: 'Febry Damatraseta Fairuz',  
  nim: '3120200907',  
  is_active: true,  
  degrees: ['S1 Teknik Informatika', 'S2 Teknologi Rekayasa Terapan']  
}
```

Value pada object bisa memiliki bentuk tipe data apapun dalam 1 key

4. Akses data menggunakan key

Dalam mengakses sebuah informasi dari object, cara yang dilakukan ialah dengan memiliki format bentuk yaitu **[variable.key]** dimana diawali dengan nama variabelnya lalu tambahkan simbol titik (.) dan diakhiri dengan nama key yang akan dipanggil.

```
lecturer.name;  
lecturer.nim;
```

Kapan menggunakan Array atau Object?

Gunakan array jika data yang dimiliki berupa daftar, memiliki urutan, data lebih dari 1. Sedangkan menggunakan Object jika dalam kondisi datanya memiliki atribut, dan setiap data memiliki identitas jelas.

2.2.6 Selection (Percabangan)

Percabangan adalah struktur kontrol dalam JavaScript yang digunakan untuk menjalankan kode berdasarkan kondisi tertentu. Dengan percabangan, program dapat mengambil keputusan yang berbeda tergantung pada nilai atau keadaan yang diberikan.

Jenis-Jenis Percabangan dalam JavaScript:

1. IF Statement

IF digunakan untuk menjalankan kode jika suatu kondisi bernilai benar (true). Sedangkan jika kondisi salah kode if tidak dijalankan.

```
const score = 80;
if (score >= 70) {
  console.log("Congratulations! You passed the exam.");
}
```

2. IF ELSE Statement

IF ELSE digunakan ketika terdapat dua kemungkinan kondisi, yaitu benar dan salah.

```
const score = 80;
if (score >= 70) {
  console.log("Congratulations! You passed the exam.");
} else {
  console.log("Sorry, you failed the exam.");
}
```

3. Nested IF

Nested IF adalah percabangan di dalam percabangan, digunakan ketika terdapat lebih dari dua kondisi.

```
const score = 80;
if (score >= 70) {
  if(score >= 90){
    confirmation("Congratulations! You passed the exam with a grade A.");
  }else{
    confirmation("Congratulations! You passed the exam with a grade B.");
  }
} else {
  console.log("Sorry, you failed the exam.");
}
```

4. Ternary Operator

Ternary operator digunakan untuk percabangan singkat dalam satu baris kode.

Syntax Short IF: kondisi ? nilaiJikaTrue : nilaiJikaFalse;

```
const score = 80;
const results = score >= 70 ? 'Passed' : 'Failed';
console.log(`Your score: ${score}, Result: ${results}`);
```

Jika ternary operator digunakan dalam JSX maka dapat diimplementasikan seperti dibawah ini:

```
export default function LatihanTernary() {
  const score = 80;
  return (
    <View style={styles.container}>
      <Text>👋 Welcome</Text>
      <Text>Praktikum Pemograman Perangkat Bergerak</Text>
      <Text>Grade: {score} >= 70 ? "Passed" : "Failed"</Text>
    </View>
  );
}
```

5. Switch Case

Switch Case digunakan untuk memilih satu blok kode dari banyak kemungkinan kondisi, berdasarkan nilai tertentu. Percabangan ini cocok digunakan ketika terdapat banyak kondisi yang membandingkan satu variabel yang sama.

```
const hari = 3;
switch (hari) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Day is not a valid day.");
}
```

2.2.7 Repetition (Perulangan)

Perulangan adalah struktur kontrol dalam pemrograman yang digunakan untuk menjalankan satu atau beberapa baris kode secara berulang selama suatu kondisi tertentu masih terpenuhi. Dalam pengembangan aplikasi React Native, looping sering digunakan untuk:

- Menampilkan daftar data (list)
- Memproses array atau object
- Menghasilkan komponen secara dinamis

Jenis-Jenis Looping dalam JavaScript:

1. For Loop

Digunakan ketika jumlah perulangan **sudah diketahui**.

```
for (let i = 1; i <= 5; i++) {
  console.log(`Looping iteration ${i}`);
}
```

2. While Loop

Digunakan ketika perulangan dilakukan selama kondisi bernilai true.

```
let i = 1;
while (i <= 3) {
  console.log("Value i:", i);
  i++;
}
```

3. Do While Loop

Mirip dengan while, tetapi kode dijalankan minimal satu kali, meskipun kondisi bernilai false.

```
let i = 5;
do {
  console.log("Nilai i:", i);
  i++;
} while (i < 5);
```

4. For ...of Loop

Digunakan untuk mengakses nilai dari array atau string.

```
const professors = ["Anton", "Isnan", "Septian"];
for (const item of professors) {
  console.log(item);
}
```

5. For ...in Loop

Digunakan untuk mengakses key (property) dari object.

```
const lecturer = {
  name: "Jemy",
  age: 70,
  degree: "S2"
};
for (const key in lecturer) {
  console.log(key + ": " + lecturer[key]);
}
```

6. Looping Array

map() digunakan untuk:

- Mengolah array

```
const data = ["TI", "SI", "MP"];
data.map((item, index) => {
  console.log(index, item);
});
```

- Menghasilkan komponen JSX

```

export default function LatihanMap() {
  const data = ["TI", "SI", "MP"];
  return (
    <View style={styles.container}>
      <Text>Concentration</Text>
      {data.map((item, index) => (
        <Text key={index}>{item}</Text>
      ))}
    </View>
  );
}

```

2.2.8 React Function Component (RFC) dan Class Component (RCC)

Dalam pengembangan aplikasi React Native, antarmuka aplikasi dibangun dari unit-unit kode yang disebut komponen (components). Ada dua pendekatan utama untuk membuat komponen: *Functional Components* dan *Class Components*. Pendekatan ini sering disingkat dalam literatur sebagai RFC dan RCC.

1. Function Components (RFC)

Function Components (RFC) adalah komponen yang ditulis sebagai fungsi JavaScript biasa yang menerima props (input) dan mengembalikan JavaScript XML (JSX) sebagai representasi antarmuka yang akan ditampilkan. Komponen ini tidak memerlukan kelas atau metode render, sehingga sintaksnya lebih singkat dan sederhana. Seiring perkembangan React, terutama sejak versi 16.8, RFC semakin direkomendasikan karena dapat menggunakan Hooks untuk mengelola state dan lifecycle, yang sebelumnya hanya dimungkinkan pada komponen kelas [4].

Contoh RFC:

RFC dengan function umum	RFC dengan format ES6
<pre> export default function Greeting() { return <Text>Welcome..</Text>; } </pre>	<pre> const Greeting = () =>{ return <Text>Welcome..</Text>; } export {Greeting} </pre>

Tabel 2.2 Perbedaan Struktur kode program antara RFC Umum dan ES6

2. Class Components (RCC)

Class Components (RCC) adalah komponen yang ditulis menggunakan kelas ES6 yang mewarisi dari *React.Component*. Komponen kelas ini memiliki metode *render()* yang mengembalikan JSX. Secara tradisional, RCC diperlukan ketika komponen harus mengelola state internal atau memanfaatkan *lifecycle methods*. Walaupun kini fungsionalitas ini telah tersedia melalui Hooks, RCC masih didukung oleh React dan dapat ditemukan dalam kode *legacy* atau pada kasus tertentu [5].

Contoh RCC:


```
class Greeting extends React.Component {  
  render() {  
    return <Text>Welcome...</Text>;  
  }  
}
```

3. Perbedaan RFC dengan RCC

Berikut ringkasan perbedaan utama antara kedua pendekatan komponen:

1. Sintaks dan Struktur
 - RFC ditulis sebagai fungsi JavaScript yang langsung mengembalikan JSX, tanpa metode *render()* atau pewarisan kelas [4].
 - RCC adalah kelas yang memperluas *React.Component* dan memerlukan metode *render()* untuk mengembalikan JSX [5].
2. Pengelolaan State
 - RFC menggunakan Hooks seperti *useState* untuk mengelola state dalam komponen[6].
 - RCC menggunakan properti *this.state* dan metode *this.setState()* untuk mengubah state[7].
3. Lifecycle Methods
 - RFC dapat menggunakan Hooks seperti *useEffect* untuk mensimulasikan perilaku *lifecycle* seperti *mount*, *update*, dan *unmount* [6].
 - RCC menggunakan metode built-in seperti *componentDidMount()*, *componentDidUpdate()*, dan *componentWillUnmount()* [4].
4. Kompleksitas dan Kode
 - RFC biasanya lebih ringkas dan mudah dibaca, cocok untuk komponen presentasional atau sederhana[4].
 - RCC dapat memiliki kode lebih panjang karena struktur kelas dan penanganan state tradisional[5].
5. Rekomendasi Industri
 - Seiring perkembangan React, RFC dengan Hooks makin menjadi *best practice* karena fleksibilitas dan kemudahan penggunaannya, sementara RCC masih didukung untuk kompatibilitas kode lama[6].

4. Kapan menggunakan RFC dan RCC

- RFC sangat cocok untuk komponen modern dan umumnya direkomendasikan karena sintaks yang sederhana dan dukungan Hook.
- RCC masih relevan untuk kode lama atau ketika diperlukan metode lifecycle tradisional tanpa Hook.

2.2.8 JavaScript XML (JSX)

JSX (*JavaScript XML*) adalah ekstensi sintaks JavaScript yang memungkinkan penulisan struktur antarmuka (UI) menggunakan format yang menyerupai HTML di dalam kode JavaScript. JSX digunakan oleh React dan React Native untuk mendeskripsikan tampilan komponen secara deklaratif dan terstruktur.

Meskipun terlihat seperti HTML, **JSX bukan HTML**, melainkan sintaks yang akan ditranspilasi menjadi JavaScript murni menggunakan alat seperti Babel sebelum dijalankan oleh aplikasi.

Peran JSX dalam React Native

Dalam React Native, JSX digunakan untuk:

- Menyusun tampilan aplikasi (*View*, *Text*, *Image*, dll.)
- Menampilkan data secara dinamis
- Mengatur logika tampilan menggunakan conditional dan looping
- Menghubungkan JavaScript dengan UI secara langsung

Dengan JSX, pengembang dapat memfokuskan pengembangan pada komponen, bukan pada manipulasi DOM atau UI secara manual.

Karakteristik JSX

- Ditulis di dalam file JavaScript atau **.jsx**
- Menggabungkan logika JavaScript dan tampilan UI
- Mendukung ekspresi JavaScript menggunakan **{ }**
- Digunakan untuk mendefinisikan struktur komponen React Native

Contoh penggunaan JSX

```
function HomeScreen() {  
  return (  
    <View>  
      <Text> Hello World 🐼 🌍!</Text>  
    </View>  
  );  
}
```

Dari kode **RFC HomeScreen** diatas, maka file yang kita simpan diharuskan menggunakan extension **.jsx**, seperti **HomeScreen.jsx** begitu sebaliknya, jika ingin mengimplementasikannya dengan RCC maka perlakuannya serupa.

Pemanggilan Variabel di dalam JSX

Dalam React Native, tampilan antarmuka ditulis menggunakan JSX (JavaScript XML). JSX memungkinkan penulisan struktur UI yang menyerupai HTML, namun tetap berada di dalam bahasa JavaScript. Oleh karena itu, ketika ingin menyisipkan nilai variabel JavaScript ke dalam JSX, diperlukan mekanisme khusus agar JSX dapat mengenali bahwa nilai tersebut berasal dari JavaScript, bukan teks biasa.

Setiap pemanggilan variabel, ekspresi, atau fungsi JavaScript di dalam JSX harus diawali dengan tanda kurung kurawal **{ }**. Tanda kurung kurawal berfungsi sebagai penanda transisi dari JSX ke JavaScript.

Contoh pemanggilan variabel dalam JSX

```

1  import React from "react";
2  import { View, Text } from "react-native";
3
4  const SampleVariableinJSX = () => {
5    const title = "Contoh penggunaan variable pada JSX";
6    const personalData = {
7      name: "Anton Sukanto",
8      jurusan: "Teknologi Informasi",
9      aktif: true,
10   };
11   const extracurricular = ["Basketball", "Robotics", "Mentoring"];
12   const a = 10, b = 20;
13
14   return (
15     <View>
16       <Text>{title}</Text>
17       <Text>Jawaban penjumlahan: {a + b}</Text>
18
19       <Text>Memanggil data array pada JSX</Text>
20       <Text>Extracurricular</Text>
21       {extracurricular.map((item, index) => (
22         <Text key={index}>- {item}</Text>
23       ))}
24
25       <Text>Memanggil data Object pada JSX</Text>
26       <Text>Fullname: {personalData.name}</Text>
27       <Text>Department: {personalData.jurusan}</Text>
28       <Text>Status: {personalData.aktif ? "Active" : "Not Active"}</Text>
29     </View>
30   );
31 };
32 export default SampleVariableinJSX;

```

2.3 Praktikum

Pada bagian ini, akan dilakukan penerapan konsep dasar JavaScript yang umum digunakan dalam pengembangan aplikasi React Native. Studi kasus difokuskan pada pengolahan dan penampilan data diri dalam bentuk *object*, daftar matakuliah dalam bentuk *array of object*, serta data hobi dalam bentuk *array*. Selain itu, turut diperkenalkan penggunaan tipe data lain seperti *boolean*, *number*, dan *string* dalam konteks aplikasi.

Melalui latihan ini, konsep percabangan digunakan untuk mengatur alur logika aplikasi, sementara perulangan dimanfaatkan untuk menampilkan data secara dinamis. Praktikum ini bertujuan untuk memberikan pemahaman awal mengenai bagaimana dasar-dasar JavaScript diimplementasikan secara langsung dalam membangun tampilan dan logika aplikasi React Native.

Pada project sebelumnya yang telah dibuat dengan nama *"MyFirst-Project-React"*, buat sebuah file baru dengan nama *Latihan2.jsx* di dalam folder *screens*. Buatlah komponen react native seperti pada code dibawah ini:

Code Latihan2.jsx

```

1  import React from "react";
2  import { View, Text, StyleSheet } from "react-native";
3
4  const Latihan2 = () => {
5    const personalData = {
6      name: "Anton Sukamto",
7      jurusan: "Teknologi Informasi",
8      aktif: true,
9    };
10   const course_lists = [
11     { id: 1, name: "Mobile Programming", code: "PPB01" },
12     { id: 2, name: "Web Programming", code: "PW04" },
13     { id: 3, name: "Calculus", code: "AC09" },
14   ];
15   const extracurricular = ["Basketball", "Robotics", "Mentoring"];
16   const total_point = 120;
17   const criteria_point = total_point >= 300;
18
19   return (
20     <View style={styles.container}>
21       <Text style={styles.title}>Personal Information</Text>
22       <Text>Fullname: {personalData.name}</Text>
23       <Text>Jurusan: {personalData.jurusan}</Text>
24       <Text>Status: {personalData.aktif ? "Active" : "Not Active"}</Text>
25
26       <Text style={styles.title}>My Courses</Text>
27       {course_lists.map((course) => (
28         <Text key={course.id}>
29           {course.code} - {course.name}
30         </Text>
31       ))}
32
33       <Text style={styles.title}>Extracurricular</Text>
34       {extracurricular.map((item, index) => (
35         <Text key={index}>- {item}</Text>
36       ))}
37
38       <Text style={styles.title}>Evaluation Criteria</Text>
39       <Text>{criteria_point ? "Eligible" : "Ineligible"}</Text>
40     </View>
41   );
42 };

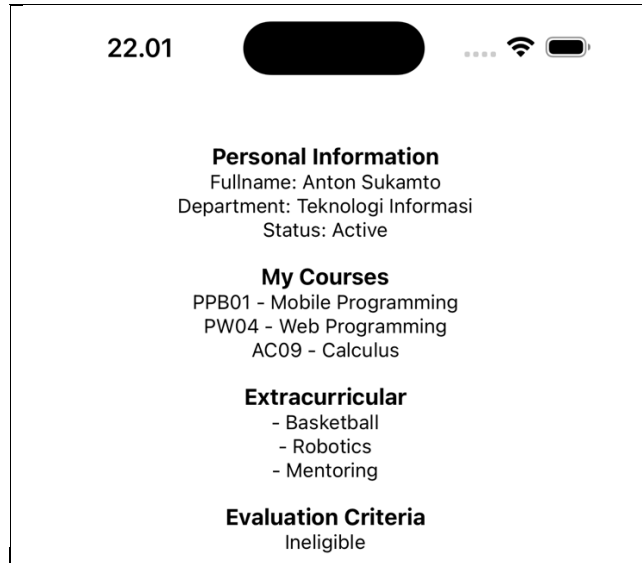
```

Output

```

43
44 const styles = StyleSheet.create({
45   container: {
46     flex: 1,
47     backgroundColor: '#fff',
48     alignItems: 'center',
49     justifyContent: 'center',
50   },
51   title: {
52     marginTop: 15,
53     fontWeight: "bold",
54     fontSize: 16,
55   },
56 });
57
58 export default Latihan2;

```



Gambar 2.1 Output Program Expo

2.4 Tugas Latihan Praktikum

Latihan praktikum pada Bab 2 difokuskan pada pengenalan lingkungan pengembangan React Native menggunakan Expo. Pembaca diarahkan untuk memahami dasar JavaScript untuk React Native seperti penggunaan variable, tipe data, operator, fungsi, array dan object, percabangan dan perulangan.

Pada latihan kali ini tidak perlu membuat project react-native baru, melainkan **melanjutkan dari project sebelumnya** (Tugas Latihan Praktikum pada BAB 1).

Latihan 1: Membuat RFC latihan ke-2

Buatlah file RFC bernama Praktikum 2, dan tampilkan tulisan "Memulai Tugas Latihan Praktikum 2". Dalam membuat komponen ini diharapkan untuk mengikuti alur navigasi yang telah disampaikan pada sub bab 1.5.4.

Latihan 2: Evaluasi Berat Badan Ideal Berdasarkan Data Diri dan Asupan Kalori

Dalam RFC Praktikum 2, buatlah tampilan untuk menghitung berat badan ideal berdasarkan Porsi Makanan Harian, dengan kondisi dibawah ini:

1. Membuat data diri dalam bentuk object yang memuat informasi nama, berat badan (kg), dan tinggi badan (cm).
2. Menyimpan data porsi makanan harian dalam bentuk array of object, yang terdiri dari nama waktu makan dan jumlah kalori.
3. Menggunakan perulangan (looping) untuk menghitung total kalori harian dari seluruh porsi makanan.
4. Melakukan konversi tinggi badan dari centimeter ke meter untuk keperluan perhitungan.
5. Menghitung *Body Mass Index* (BMI) menggunakan berat badan dan tinggi badan.
6. Menerapkan percabangan (conditional) untuk menentukan status BMI (kurus, ideal, atau berlebih).
7. Menggunakan percabangan untuk menentukan kategori asupan kalori harian (kurang, cukup, atau berlebih).

8. Membuat logika kesimpulan yang mengevaluasi kesesuaian antara status BMI dan asupan kalori.
9. Menampilkan seluruh data dan hasil perhitungan ke dalam tampilan aplikasi React Native menggunakan komponen Text.
10. Memastikan kode ditulis secara terstruktur, mudah dibaca, dan diberi komentar seperlunya.

Output:

- Informasi data diri
- Daftar porsi makanan harian
- Total kalori harian
- Nilai BMI
- Status berat badan
- Status asupan kalori
- Kesimpulan akhir evaluasi

Latihan 3: Merubah file entry point

Ubahlah file index.jsx dalam folder app untuk merubah entry point dan arahkan kedalam Praktikum 2 untuk melihat tampilan output aplikasi.



Gambar 2.2 Tampilan Output Program

Latihan 4: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project react expo yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository **"PPB-Praktikum-NPM"**, namun *commit* dengan nama **"Tugas Praktikum-2"**.
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).